

ZIREN 2.0: A High-Performance MIPS Zero-Knowledge Virtual Machine with GPU-Accelerated BaseFold Recursion

The ZKM Project

May 22, 2026

Abstract

We present ZIREN 2.0, the second-generation iteration of the ZIREN zero-knowledge virtual machine (zkVM) over the MIPS32r2 instruction set architecture. ZIREN 2.0 replaces its previous Fast Reed–Solomon Interactive Oracle Proof (FRI) backend with a hierarchical polynomial commitment construction combining **BaseFold** over the **KoalaBear** prime field with **Jagged** sumcheck reduction for variable-height chip traces and **Stacked** batching for heterogeneous rounds. We retire the cross-shard duplicate commitment used in earlier revisions and migrate to a sumcheck-binding architecture in which the soundness role of permutation and quotient commitments is folded into per-chip **LogUp-GKR** lookup arguments and per-shard zerocheck. The recursion layer is rewritten around four phases (transcript prologue, **LogUp-GKR** replay, zerocheck replay, **Jagged** opening) and shipped together with a GPU prover targeting NVIDIA’s RTX 5090 family. The just-in-time MIPS executor delivers bit-exact trace generation at roughly six times the rate of the interpreter on trace-mode execution. End-to-end on a representative Ethereum execution-layer block (the **reth** workload at ~ 420 million cycles) the system produces a Groth16-ready BN254 wrap proof in 94.1 seconds on eight GPUs, with verified correctness through every recursion layer. We describe the architectural shifts, the protocol-level adjustments required for soundness under the new commitment shape, and a multi-workload performance evaluation. We also document known limitations, including remaining gaps in the in-circuit verifier’s algebraic alignment between CPU and GPU prover paths.

1 Introduction

Zero-knowledge virtual machines (zkVMs) compile execution traces of general-purpose programs into succinct cryptographic proofs that the trace was produced by a particular program on particular inputs and consumed at most a stated number of cycles. The recent generation of practical zkVMs descends from the architectural template established by SP1 [18]: a STARK-based prover over a small prime field, with chip-decomposed AIR constraints, recursive aggregation, and a Groth16 outer wrap on the BN254 curve to produce verifier-cheap proofs for on-chain consumption.

ZIREN [21] is a zkVM in this lineage whose guest instruction set is MIPS32r2 rather than RISC-V32IM. The choice of MIPS is motivated by the maturity of its instruction encoding, the directly available bit-manipulation and sign-extension primitives (notably **SEB**, **SEH**, **WSBH**, **EXT**, **INS**), and the existing deployment of MIPS in production fraud-proof systems such as Optimism’s MIPS interpreter for fault proofs.

This paper describes ZIREN 2.0, a substantial redesign of the prover and recursion stack carried out across approximately fourteen months of incremental development. The first generation of ZIREN shared SP1’s earlier proof-system organisation: a four-batch commitment structure (preprocessed, main, permutation, quotient) under FRI [3], with cross-shard binding implemented via

duplicate commitments. The second generation moves to a hierarchical commitment construction — **BaseFold** [22] for per-stripe encoding, **Stacked** batching over rounds, and **Jagged** sumcheck reduction across variable-height chip traces — and folds the soundness role of the legacy auxiliary commitments into per-chip sumcheck bindings: **LogUp-GKR** lookup proofs [13] for cross-table arguments and zerocheck for per-chip AIR constraint satisfaction.

1.1 Problem statement and contributions

The previous ZIREN proof system suffered from three structural issues that this work resolves.

Memory pressure. The legacy FRI-based prover materialised the dense polynomial low-degree extension (LDE) for the entire shard in host memory before committing. For multi-million-cycle workloads such as Tendermint light-block verification and Ethereum execution-layer blocks, the working set exceeded 100 GB on standard hardware, producing out-of-memory failures on otherwise feasible inputs. The hierarchical **BaseFold** + **Stacked** + **Jagged** stack encodes stripes incrementally so that peak memory is bounded by stripe size, not by whole-vector LDE size.

Verifier complexity. The cross-shard duplicate-commitment idiom required reconciling two parallel views of the same data; downstream removal of the duplicate commitment, while preserving Fiat–Shamir [6] soundness, required a careful sumcheck-binding rewrite. We complete this rewrite end-to-end and describe how the binding is recovered through a transcript-observation discipline and per-chip cumulative-sum carriage.

Hardware throughput. A reference CPU implementation of even a modest proving stack on tens-of-millions-of-cycles workloads is a thirty-minute proposition. Production deployment of a zkVM requires hardware acceleration. We present a GPU prover stack with device-resident proof state, asynchronous per-shard streams, a polynomial-encoding cache, and pipelined compression that delivers a near-linear speedup curve up to four GPUs and a saturated four-eight GPU range on representative workloads.

The principal contributions of this paper are:

1. A protocol-level description of the post-migration ZIREN prover, including the precise role of **BaseFold** as a fold-based polynomial commitment scheme over **KoalaBear**, the **Stacked** wrapper for heterogeneous-round batching, and the **Jagged** sumcheck for variable-height chip traces (§3).
2. A description of two structurally distinct prover code paths — an SP1-aligned path and a ZIREN-native path — and the conditions under which they produce verifier-compatible proofs (§4).
3. A discussion of subtle soundness corrections required during the migration, including a per-chip-padded log-degree convention for dummy proofs in the recursion verifier and a fold-orientation tag on the proof to support path-aware verification while the inner sumcheck algebra is being aligned across implementations (§4).
4. A multi-workload performance evaluation over a Fibonacci sweep, JSON parsing, a chess engine, Tendermint light-block verification, and Ethereum block proving via **reth** and **geth**, on hardware spanning a single GPU through to eight (§5).

5. A description of the just-in-time (JIT) MIPS executor and its differential validation against the reference interpreter (§4).

1.2 Roadmap

Section 2 reviews the cryptographic background: STARKs, polynomial commitment schemes, sumcheck-based interactive arguments, lookup arguments, and the recursion structure inherited from SP1. Section 3 describes the ZIREN 2.0 architecture proper. Section 4 covers the implementation choices that shaped the system. Section 5 presents the performance evaluation. Section 6 discusses design rationale, limitations, and avenues for future work. Section 7 situates the work in the broader zkVM landscape, and Section 8 concludes.

2 Background

This section reviews the cryptographic and architectural ingredients underlying ZIREN 2.0. We assume familiarity with finite-field arithmetic and Merkle-tree commitments and refer the reader to standard treatments for foundational material.

2.1 Zero-knowledge virtual machines

A zkVM is a system in which a *guest program* expressed in a target instruction-set architecture is executed on *guest inputs*, producing a *guest trace* together with a succinct, soundly verifiable proof that the trace was produced honestly by that program on those inputs. The proof is succinct in the sense that its size and verification time are independent of (or only mildly dependent on) the trace length.

Concretely, the prover takes an executable in the target ISA, runs it under a recording interpreter or compiler-instrumented executor, and emits a sequence of structured trace tables (one per chip). Each chip table is a width- w matrix over the prime field $\mathbb{F} = \text{KoalaBear}$, where `KoalaBear` is the Koala Bear prime field used by the Plonky3 ecosystem [15]. The constraints on those tables are encoded as an algebraic intermediate representation (AIR) that the prover demonstrates to be satisfied via a polynomial commitment scheme (PCS).

2.2 MIPS32r2 versus RISC-V

The choice of guest ISA influences the chip set, the precompile boundary, and the interaction with existing developer tooling. ZIREN targets MIPS32r2 rather than the RV32IM target used by SP1. The structural similarities (32 general-purpose registers, fixed-width instructions, similar opcode counts) mean that significant portions of the prover architecture transfer directly. Notable differences relevant to zkVM engineering are:

- MIPS has explicit `HI/L0` registers receiving the upper and lower halves of multiplication and division outputs.
- MIPS uses a branch-delay-slot semantics, in which the instruction immediately following a branch executes before the branch takes effect; the executor must track `next_pc` and `next_next_pc` separately.
- MIPS provides bit-manipulation primitives (`WSBH`, `EXT`, `INS`) that simplify the encoding of common hashing and byte-shuffling sequences.

2.3 STARKs and polynomial commitment schemes

The zkVM prover converts each chip table into a multilinear polynomial (MLE) over the boolean hypercube $\{0, 1\}^n$ where $n = \lceil \log_2 \text{height} \rceil$, commits to a low-degree extension of each MLE via a polynomial commitment scheme, and proves AIR satisfaction by combining the commitments with sumcheck-style protocols [11]. Two PCS families dominate practical small-field zkVM construction:

Fast Reed–Solomon IOPP (FRI). FRI [3] commits to the Reed–Solomon encoding of a polynomial over a multiplicative subgroup and uses repeated random folding to reduce a low-degree claim to a constant-size statement. FRI is the workhorse of the original Plonky3 and SP1 stacks.

BaseFold. BaseFold [22] is a multilinear polynomial commitment scheme that interleaves random linear combinations from the sumcheck protocol with FRI-style codeword folding. Compared to a naive FRI commitment to a flattened MLE, BaseFold avoids a wasted univariate-to-multilinear conversion and exposes finer-grained streaming opportunities, enabling per-stripe encoding rather than whole-vector LDE.

We use BaseFold throughout ZIREN 2.0; the FRI baseline is retained only in the legacy outer-shrink path that does not yet emit a CPU chip.

2.4 Sumcheck-based lookup arguments

Modern small-field zkVMs rely heavily on lookup arguments for cross-chip constraints (e.g., the CPU chip asserts the validity of an instruction by looking up its decomposed form in a program-image chip). ZIREN 2.0 uses the LogUp-GKR construction of Habök, Eagen, and Pawelczak [13], in which each lookup is reduced to a product-argument over rational functions and discharged via a GKR-style layered sumcheck [8].

A LogUp-GKR proof for a chip produces a vector of layer proofs descending from the GKR circuit root to its leaves; the protocol is shard-local in our integration, so a multi-shard workload produces one LogUp-GKR proof per chip per shard.

2.5 Zerocheck

The *zerocheck* protocol reduces “polynomial f vanishes on a domain D ” to “ f evaluated at a random point r equals the expected sumcheck value.” ZIREN 2.0’s prover discharges all AIR constraints via a single zerocheck per chip, batched into a per-shard sumcheck after a transcript-mixed random linear combination over chips.

2.6 Jagged polynomial commitment

Real zkVM chip tables have non-uniform heights: the CPU chip might be at 2^{20} rows while a precompile chip is at 2^{12} . The Jagged construction (originating in SP1’s `slop` crate family) reduces a heterogeneous batch of MLEs of differing heights to a single sumcheck claim that can then be discharged against a stacked BaseFold commitment.

Formally, given chip MLEs $f_1, \dots, f_k$ of heights $2^{n_1}, \dots, 2^{n_k}$, the verifier samples random combination weights α_i and evaluation points z_i . The prover demonstrates $\sum_i \alpha_i f_i(z_i)$ equals the claimed value via a structured sumcheck whose variable ordering encodes the per-chip prefix-sum heights. The *jagged-eval* sub-protocol checks the consistency of the claimed per-chip evaluations against the public chip-height vector; in ZIREN 2.0 this is discharged via a small DSL fragment running a 4×16 branching-program evaluation [2] together with a Horner-form prefix-sum check.

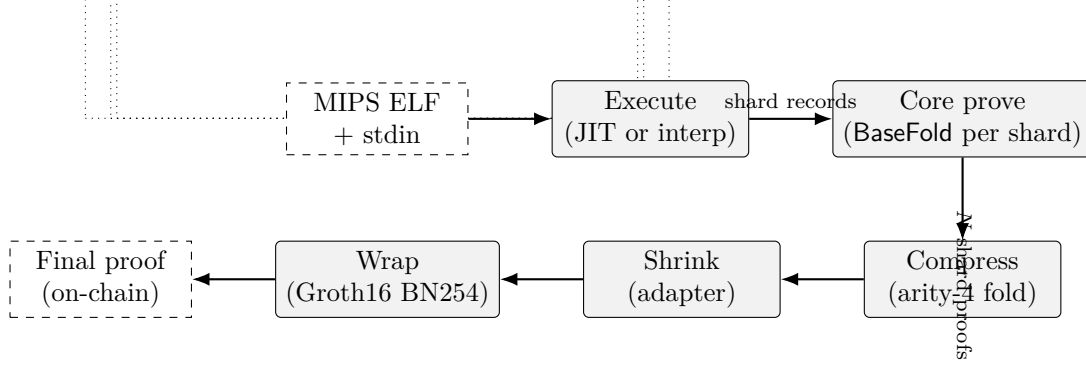


Figure 1: The ZIREN 2.0 proving pipeline. The executor produces shard records and chip event lists; the core stage emits one **BaseFold** shard proof per shard; the compress stage recursively aggregates pairs of shard proofs; the shrink stage adapts the final aggregate to the wrap circuit; the wrap stage produces a Groth16 proof on BN254.

2.7 Recursion and outer wrap

zkVM proofs are produced in two recursion layers and then wrapped into an outer SNARK. The prover first emits per-shard proofs (the *core* layer). A *compose tree* then aggregates pairs of shard proofs into combined proofs, halving the active proof count at each level until one proof remains (*compress*). A final *shrink* layer adapts the compressed proof to a smaller proof system, and a *wrap* stage produces a Groth16 proof on the BN254 curve via **gnark** [4].

The recursion layers are themselves zkVMs in miniature: each verifies a smaller fixed-shape program (a “recursion program”) compiled to a DSL of register-allocated arithmetic operations, executed on a tiny recursion runtime, and proved with a STARK over the same Koala Bear prime field. The recursion verifier consumes a previous proof together with a verifying key, dispatching among a small set of program shapes via a *verifying key map* (VK map) that records the hash of each admissible verifying key.

2.8 Inherited architecture from SP1

ZIREN originated as an SP1 fork retargeted to MIPS, and the second-generation migration was steered against periodic comparison with the SP1 reference. We acknowledge throughout that the protocol templates (chip decomposition, recursion structure, jagged sumcheck, basefold late binding, in-circuit verifier organisation) follow SP1’s design. The contributions of ZIREN 2.0 that this paper foregrounds are concentrated in: the MIPS-specific executor and JIT, the GPU prover’s residency and pipelining choices, the post-cross-binding sumcheck rewrite, and the integration discipline that brings the protocol stack to the production performance levels reported.

3 Architecture

ZIREN 2.0 is organised as a four-stage pipeline: *execute and shard*, *core prove*, *compress*, *shrink*, and *wrap*. The execute stage produces shard-sized chunks of trace records and chip event lists. The core prove stage commits per-shard MLEs and emits per-shard **BaseFold** proofs. The compress stage recursively aggregates these into a single proof. The shrink and wrap stages convert the aggregate proof into a Groth16-on-BN254 outer proof. Figure 1 summarises the flow.

3.1 The shard-level BaseFold shard proof

A shard proof in ZIREN 2.0 is a single structured object containing the per-shard public values, a single main-trace commitment, the per-chip LogUp-GKR proofs, the per-chip zerocheck proofs, the per-chip opened values (main and preprocessed), and a serialised Jagged-BaseFold opening proof. Crucially, in the post-migration architecture the auxiliary commitment vector (which in earlier prover incarnations carried separate permutation and quotient commitments) is empty: the soundness role previously played by those commitments is now discharged by the sumcheck bindings emitted by the LogUp-GKR and zerocheck protocols, with the quotient terms folded into the FRI commit inside BaseFold.

3.1.1 Transcript binding

The shard transcript is initialised by observing the public values, the main commitment, and per-chip metadata (chip name, chip log-height). The verifier then samples LogUp-GKR challenges and discharges all per-chip lookup proofs in a uniform order, observing each layer’s claim before sampling the next descent challenge. Zerocheck follows, again per-chip in a uniform order. The Jagged opening point is sampled from the post-zerocheck transcript state; the Jagged verifier reduces the per-chip evaluation claims to a single Stacked opening claim, which is then discharged via BaseFold.

3.1.2 The per-chip-padded log-degree convention

A subtle invariant runs through the dummy-proof generators that allocate witness slices for the recursion verifier at compile time. Concretely, the recursion circuit requires an upper bound on the total length of witness data it will consume, and a naive bound based on per-chip lookup counts under-allocates whenever a chip has a non-power-of-two interaction count. We resolve this by padding each chip’s lookup count to the next power of two before summation, and rounding the total up to the next power of two:

$$W_{\max} = 2^{\lceil \log_2 \sum_c \text{nextpow2}(\ell_c) \rceil + 1}, \quad (1)$$

where ℓ_c denotes the number of cross-table lookups on chip c . This per-chip-padded summation matches the structure of the prover’s actual witness emission, where each chip’s lookup proof is independently padded; without it, the recursion circuit silently allocates a buffer of the wrong size and the verifier panics at the first round of LogUp-GKR.

3.2 BaseFold polynomial commitment over KoalaBear

For a multilinear polynomial $f \in \mathbb{F}[X_1, \dots, X_n]$ given as evaluations on $\{0, 1\}^n$, the BaseFold commit consists of a Reed–Solomon encoding of f at the coset of order $2^{n+\log \rho^{-1}}$, committed via a Merkle tree over Poseidon2 digests. The opening protocol at a point $\mathbf{z} = (z_1, \dots, z_n) \in \mathbb{F}_{p^d}^n$ proceeds in n rounds, each consisting of:

- An MLE-fold step: $f_{r+1}(X_2, \dots, X_{n-r}) = (1 - \beta_r)f_r(0, X_2, \dots) + \beta_r f_r(1, X_2, \dots)$ where β_r is a sumcheck round challenge.
- A codeword-fold step on the committed Reed–Solomon encoding: even and odd indices are paired and combined under β_r to yield the encoding of f_{r+1} .
- A Merkle commit to the folded codeword that will be queried in the final phase.

The single *key invariant* of **BaseFold** is that, throughout the round descent, the codeword’s K -th fold equals the MLE’s K -th fold, so the same random challenge can be reused as both the sumcheck reduction challenge and the FRI fold parameter. After n rounds, the codeword is a constant; this constant is asserted to equal $f(\mathbf{z})$ minus the accumulated contributions from earlier sumcheck rounds. A query phase then samples q random codeword positions and verifies that each is consistent with the committed root via Merkle inclusion paths.

We use the Koala Bear prime field $\mathbb{F}$ (with $|\mathbb{F}| = 2^{31} - 2^{24} + 1$, a 31-bit prime) for the base field, with a degree-4 extension $\mathbb{F}_{p^4}$ for protocol challenges. The two-adicity of KoalaBear supports the Reed–Solomon coset structure used by the encoding step. Hashes use the Poseidon22 permutation over Koala Bear, with sponge mode for variable-length absorption and truncation-based compression for the Merkle tree.

3.2.1 Soundness parameters

ZIREN 2.0 ships two soundness configurations. A development configuration uses code rate $1/2$ ($\log \rho^{-1} = 1$) with no grinding for fast iteration; this yields approximately 58 proven bits of soundness under the Johnson bound. The production configuration uses rate $1/16$ ($\log \rho^{-1} = 4$) with 16 bits of Poseidon22-based proof-of-work grinding on each round’s transcript binding, for approximately 118 proven bits, exceeding the 100-bit aggregate target.

3.3 Stacked batching

The shard proof presents multiple commitment rounds: at minimum, a preprocessed-trace round (committed once, before the prover ever sees the witness) and a main-trace round (committed per shard). In some configurations, additional auxiliary rounds appear. The **Stacked** wrapper batches commitments of differing widths and per-round MLE counts into a single **BaseFold** commitment by interleaving MLEs at a chosen stacking height and committing the resulting wider matrix.

Given a stacking height h chosen via the heuristic $h^* = \min(8, \lfloor \log_2 N/2 \rfloor)$ for total commit volume N , the prover lays out the input MLEs as horizontal stripes of height h^* , padding each MLE to the nearest multiple of h^* rows. The committed object is a single matrix of height h^* rows by “stacked” columns wide. Opening at a point $\mathbf{z}$ splits $\mathbf{z} = (\mathbf{z}_{\text{batch}}, \mathbf{z}_{\text{stack}})$, with $\mathbf{z}_{\text{stack}}$ of length h^* used to evaluate within a stripe and $\mathbf{z}_{\text{batch}}$ used to select among the stacked stripes.

A critical correctness invariant of **Stacked** is that the committed data length be a power of two and that $|\mathbf{z}| = \log_2(\text{committed length})$. Violating this invariant produces the proof-shape mismatches discussed in §4.4.

3.4 Jagged sumcheck reduction

The **Jagged** sumcheck reduces a batch of MLE evaluation claims at differing heights (f_c, n_c, z_c) to a single **Stacked** evaluation claim. The reduction proceeds by:

1. Sampling a random linear combination $\alpha_1, \dots, \alpha_k$ over the k per-chip claims.
2. Running a sumcheck over $\max_c n_c$ variables on the combined polynomial $G(\mathbf{X}) = \sum_c \alpha_c f_c(\mathbf{X}|_{n_c}) \cdot \text{geq}_c(\mathbf{X})$, where geq_c is a polynomial that is 1 on the actual support of chip c and corrects for the height differential.
3. Producing a final claim $G(\mathbf{z}^*)$ at the sumcheck reduction point $\mathbf{z}^*$, which the verifier checks against a **Stacked** opening together with an auxiliary *jagged-eval* proof.

The jagged-eval sub-proof attests that the geq_c values used in the sumcheck combination match the public chip-height vector. ZIREN 2.0 ships a sumcheck-based jagged-eval evaluator that uses two recursion-compiler primitives: a branching-program evaluation over a compile-time-constant 4×16 transition table, and a per-prefix-sum Horner reduction with witness felts.

The per-round zero-column padding emitted by the prover must be reflected in the verifier’s `col_prefix_sums` structure; this is an easily-overlooked invariant whose violation produces dimension-mismatch panics during proof verification, as discussed in §4.4.

3.5 LogUp-GKR lookup arguments

Each chip in ZIREN 2.0 emits a LogUp-GKR proof per shard. The protocol uses a Habök-style log-derivative formulation: each lookup (requesting chip, providing chip, tuple) contributes a rational term, and the verifier checks the rational identity using a GKR circuit. The GKR circuit descends layer by layer, with each layer’s claim discharged by a sumcheck whose round challenges feed the next layer.

In the shard-level shape used by ZIREN 2.0, the prover emits one LogUp-GKR proof per chip per shard, in a layer-based descent serialisation. The recursion-circuit verifier was previously organised around a per-shard sumcheck-stack serialisation, and the proof-shape divergence between the two has been the long pole of the recursion verifier port. We discuss the divergence and the migration plan in §4.11.

3.6 GPU prover

ZIREN 2.0’s GPU prover is implemented as a separate Rust crate (`ziren-gpu`) that depends on a CUDA toolkit and a small set of hand-written CUDA kernels. The host `ziren-stark` crate dispatches polynomial-commitment work to the GPU when its `gpu` feature is enabled.

3.6.1 Device residency

The defining architectural choice of the GPU prover is *device residency*: trace tables, MLEs, codewords, and Merkle digests live on GPU memory for the duration of the per-shard proof. Host memory is used only to receive the final serialised proof bytes and to drive the high-level orchestration. This avoids the PCIe-bandwidth bottleneck of constantly shuttling intermediate state across the host/device boundary; it also concentrates memory pressure where the workload is most parallelisable.

A device-resident dense jagged trace MLE structure plays the role of SP1’s `TraceDenseData`: a single contiguous device buffer of base-field elements, indexed by per-chip offsets and column counts. The buffer is materialised once per shard from the trace tables and then reused across the commit, sumcheck, and opening phases.

3.6.2 Per-slot streams and the work-stealing pool

Multi-GPU dispatch is organised around a *worker context* per GPU, each holding a CUDA stream pool and a per-slot scratch arena. A bounded-capacity channel passes (record, trace) tuples to the worker pool; each worker pulls a tuple, runs setup, commit, and open on a free GPU, and blocks on the result before pulling the next tuple. The number of concurrent in-flight workers per GPU is configurable but, for the production workloads measured in §5, a single in-flight task per GPU was found to be optimal: oversubscription doubles the per-shard memory footprint and triggers GPU-side allocator pressure that erases the latency-hiding win.

The number of *prover-side* concurrent workers (the `shard_batch_size`) is sized as $\max(4, \min(8, 2 \cdot G))$ where G is the number of GPUs visible to the process. This scaling matches the CPU-side trace-generation throughput to the GPU-side prove throughput on our reference hardware; further oversubscription saturates the host-side state-mutex contention.

3.6.3 Verifying-key cache (V3)

Adjacent shards in the compress pipeline often share program shapes, in which case they share a verifying key. The verifying-key cache keeps the recursion-program verifying key resident on the GPU keyed by the program’s Arc pointer identity, sharing across structurally-equivalent shards. On a representative Ethereum block this yields a $\sim 60\%$ hit rate and saves an estimated three to six seconds of compress wall time per workload.

3.6.4 Pipelined compress

The compress stage benefits from a per-height pending list and an arity-4 binary tree-reduce strategy: groups of four compressed proofs are combined at each level rather than the binary arity-2 fold used in earlier revisions. This change closes a $4\times$ -to- $8\times$ GPU scaling cliff that we observed during initial multi-GPU bring-up, in which compress at 8 GPUs was no faster than at 4 GPUs because the level-by-level binary fold starved the available pool.

3.7 Recursion circuit and the in-circuit verifier

The recursion circuit is the in-DSL implementation of the verifier. It consumes a previous-layer proof and a verifying key and emits a structured assertion log that the recursion-program AIR enforces. The circuit is compiled to the recursion DSL, which is then assembled into a recursion program that the recursion runtime executes; the resulting trace is itself proven via a STARK over KoalaBear, producing the next-layer proof.

The recursion verifier is organised in four phases: the transcript prologue, LogUp-GKR replay, zerocheck replay, and Jagged opening. Each phase consumes the witness fields it requires and emits assertions whose satisfaction implies, by the soundness of the underlying protocols, that the input proof was honestly produced.

3.7.1 Lift programs, compose programs, and dummy proof allocators

ZIREN 2.0 uses several compile-time-constructed recursion programs: a *lift program* per shape that converts a shard proof into the in-circuit format expected by the recursion verifier; a *compose program* per tree-arity that combines compressed proofs at a given level; and *dummy proof allocators* that build structurally-valid but content-empty proofs to drive the compile-time AST construction of the verifier.

The dummy proof allocators are particularly performance-sensitive because the recursion-program build runs synthesis once per dummy at pre-warm time, and the original synthesis path internally called the real per-shard prove function on a synthetic input, costing tens of seconds per arity. We replaced this with a structural stub modelled on SP1’s pattern, in which the dummy proof is a struct populated with shape-correct but content-meaningless data. The per-arity dummy synthesis dropped from approximately fifteen seconds to under two hundred microseconds, reducing total prover pre-warm from 64.8 s to 2.0 s.

3.8 Just-in-time MIPS executor

The MIPS executor is a single-threaded `while not done { execute_cycle() }` interpreter in the reference build, with per-instruction overhead dominated by register-file lookup, page-table walks for memory operations, and event-list appends for trace generation.

For Linux x86_64 hosts, ZIREN 2.0 ships a just-in-time compiler that emits native x86_64 code from the MIPS instruction stream. The JIT uses `dynasm-rt` for assembly emission, pins MIPS GPRs to XMM register halves, and lowers each MIPS opcode to between one and ten x86 instructions. Loads and stores are inlined with a $\text{addr} \mapsto (\text{addr} \& \sim 7) \ll 1 + (\text{addr} \& 7)$ shift trick that places the per-word “last-write clock” adjacent to the word value in a contiguous host buffer, avoiding the BTreeMap-walking page-table lookups of the interpreter.

A bit-exactness CI guard drives the JIT and the interpreter on a random program corpus and asserts byte-identical trace event streams; this discipline catches any soundness-relevant divergence at PR submission time.

The JIT is build-time gated to Linux x86_64 release builds; other platforms compile out the JIT entirely and use the interpreter, with no runtime overhead deciding which to use.

3.9 Verifying-key map

For proofs that the same recursion program shape can be loaded from a deterministic table, ZIREN 2.0 maintains a *verifying-key map* (VK map): a file mapping the hash of each admissible verifying key to its preprocessed-trace digest. When `VERIFY_VK=true`, the verifier asserts that the loaded VK is in the map. The map is regenerated workload-driven (only program shapes actually encountered are emitted), with per-shape parallel compilation and an off-by-one fix in the first-layer enumerator to ensure shapes whose terminal layer matches the first layer are not silently skipped.

4 Implementation highlights

This section discusses the implementation-level choices that shaped the system’s performance and soundness profile. Several of these were not anticipated at the start of the migration and emerged through measurement and bug-driven analysis; we record them here both as documentation of the system’s current behaviour and as guidance for similar projects.

4.1 Two structurally distinct prover paths

ZIREN 2.0 carries two prover code paths that produce structurally distinct proofs: an *SP1-aligned* path that mirrors the reference SP1 prover’s packed-pool integration for `LogUp-GKR` and zerocheck, and a *ZIREN-native* path that descends from the prior ZIREN architecture’s per-chip dispatch.

Both paths are kept compiled and reachable; an internal dispatch decides at proof-generation time which path executes. The choice of default is significant: we observed that the SP1-aligned packed-pool path produces proofs that, at the first round of `LogUp-GKR`, diverge from the ZIREN-native path’s proofs in their inner sumcheck algebra. The divergence is not visible at the integration level (both paths produce a proof; both proofs verify under their own verifier) but is visible across the verifier boundary: a proof from the SP1-aligned path does not verify under the ZIREN-native verifier, and vice versa.

Pending the alignment of the inner sumcheck algebra, the production default routes through the ZIREN-native dispatch, which maintains end-to-end soundness in the recursion layers we currently exercise. The SP1-aligned path remains available behind a debug environment variable for kernel-set parity testing.

4.2 Witness padding and dummy proof allocators

The recursion-circuit verifier consumes its previous-layer proof through a witness stream: a sequence of KoalaBear field elements deserialised in order matching the verifier’s request pattern. The recursion program is compiled against a *dummy proof* that has the correct structural shape but content-empty data; the witness slice sizes baked into the recursion program at compile time are derived from the dummy proof’s serialised size.

A naive bound on the witness slice size for the per-chip LogUp-GKR portion underestimates the prover’s actual emission whenever a chip’s lookup count is not a power of two: the prover pads each chip’s lookup count up to the next power of two independently, and the total witness size therefore reflects per-chip-padded summation, not a single rounding of the gross total. We resolved this by adopting the per-chip-padded summation convention described in §3, both in the dummy proof allocator and in the recursion-program compile-time slice-size derivation.

The cost of getting this wrong is severe: the recursion circuit silently allocates a witness slice of the wrong size, and the verifier panics at the first round of LogUp-GKR. The cost of getting it right is a single additional `nextpow2` in the size computation.

4.3 Per-chip-padded sum in BaseFold commit hashing

A related correction lies in the BaseFold commit hash sponge: the number of LogUp-GKR interactions absorbed in the transcript must match between prover and verifier. The shard’s interaction count was, in earlier revisions, computed as a sum over chips of the per-chip raw count, but the prover-side hashing was padding per-chip first and then summing. The correction is to compute the verifier’s count as $\sum_c \text{nextpow2}(\ell_c)$ rather than $\text{nextpow2}(\sum_c \ell_c)$.

4.4 Lift normalisation and compose-cache safety

The compress pipeline aggregates shard proofs into compose-tree input via a *lift* step that converts the shard’s BaseFold proof into a normalised input for the compose recursion program. Because compose-tree caching keys on the recursion-program AST hash, lifts whose chip-set or chip-shape varies across shards produce divergent AST hashes and defeat the cache.

We force a single canonical shape on the lift output by fixing the per-chip widths at canonical values (memory-constraint chip at $\log_2 = 19$, base-ALU and extension-ALU chips at $\log_2 = 20$). With this shape fix, the compose-tree cache is safe to enable across mixed shard workloads: every lift produces the same AST hash, and the cache delivers a real wall-time reduction without invalidating downstream verification.

This fix is itself bounded by the *stacked-PCS dimension invariant*. A second class of bug in the same area arises when the per-round zero-column padding in the prover’s Jagged parameter assembly is not reflected in the verifier’s expected column-prefix-sums vector. The verifier then samples a sumcheck point of one length while the flattened batch-evaluation vector reflects another length, and the column-claims assertion fires with a $2\times$ mismatch. The correction is on the prover side, in the construction of the Jagged little-polynomial prover parameters: the per-round zero-column row counts must be included in the flat `row_counts` vector before the prefix sums are taken.

4.5 Verifying-key map enumeration and workload-driven injection

The verifying-key map’s contents must cover every program shape encountered by every workload, and a naive enumeration of all shapes in a wide range produces many thousands of entries the vast majority of which are never used. We use an SP1-style consecutive-integer enumeration of shape

parameters from 1 through 12, which covers all production workloads observed in our test suite, deduplicating to approximately 41 real shapes after structural canonicalisation.

For one-off injection (e.g., capturing a new precompile cluster’s verifying key without a full map regeneration), a workload-driven capture tool runs the prover up to the recursion-program compile point on a target ELF, extracts the resulting verifying keys, and emits them in the map format. Subsequent merging with the existing map is by hash injection.

The full map regeneration runs in approximately thirteen minutes on a moderately-provisioned host, parallelised over CPU cores with a per-worker recursion-program compile budget controlled to avoid OOM under contention.

4.6 Fold-orientation tag and path-aware verification

The two prover paths discussed in §4.1 differ in the orientation of their **BaseFold** fold step: one path’s first round folds against the most-significant bit of the evaluation point, while the other folds against the least-significant bit. The orientations are not interconvertible at the verifier level without a corresponding algebraic adjustment.

We introduce a one-bit *fold-orientation tag* on the proof, allowing path-aware verifier dispatch. The tag isolates the protocol-level invariant that distinguishes the two paths, and a resolver in the verifier reorders the evaluation point accordingly before initiating the round-by-round fold replay. This is sufficient for proof-shape compatibility (a path-A proof verifies under the path-A verifier; a path-B proof verifies under the path-B verifier), but does not yet provide full transcript compatibility across CPU and GPU verifier implementations of the same path. Full compatibility awaits the inner-sumcheck algebra alignment that is open work.

4.7 Pipelined compress and shard-batch sizing

The pipelined compress stage’s wall time on multi-GPU is sensitive to a single parameter, the `shard_batch_size`, which controls the number of concurrent prover-submit threads in the compress dispatcher. Through a parameter sweep at one through eight GPUs we identified the rule

$$\text{SBS}^* = \max(4, \min(8, 2G)), \tag{2}$$

where G is the visible GPU count. At one and two GPUs, the conservative value of 4 avoids out-of-memory failures from oversubscription on real `reth` shards. At four GPUs, $\text{SBS} = 8$ provides $2\times$ oversubscription per GPU, hiding the per-shard CPU preparation latency. At eight GPUs, $\text{SBS} = 8$ is exactly 1 : 1 with the pool. Going further to $\text{SBS} = 12$ at eight GPUs plateaus and then regresses, as worker threads contend on host-side state mutexes that the core stage uses for shared shard records.

4.8 Turn-based synchronisation: dropped

An earlier revision of ZIREN 2.0’s compress pipeline used a `TurnBasedSync` barrier primitive to enforce strict commit-ordering across worker threads. We measured this barrier and found it cost about seven percent on `reth` multi-GPU wall time and six percent on `geth`, with no soundness benefit (the ordering it enforced was already implied by data dependencies elsewhere). The SP1 reference retains the data structure but never invokes it; we removed our active usage entirely.

4.9 Just-in-time MIPS executor: parity and bring-up

The JIT MIPS executor described in §3.8 is the third generation of execute-path acceleration in ZIREN. It supersedes both the legacy interpreter (still retained as a fallback) and an earlier Cranelift-targeted experiment that was retargeted to native code emission for the inner loop.

Bringing the JIT to bit-exact parity with the interpreter on real Ziren guests required resolving thirteen distinct semantic divergences in the lowering of MIPS opcodes (rotation right, the LOCATION pseudo-instruction, HI/LO ordering, the heap-break syscall, EXT/INS, the 32-bit forms of MULT/MUL, the per-syscall register refresh order, the per-syscall trace synchronisation cadence, sign-extension of immediate offsets, the BLTZ/BGEZ branch family, and the trap-equal TEQ form). Each was identified by a differential test that ran the JIT-produced trace and the interpreter-produced trace through a byte-identity check on register state, memory state, and event ordering. Once resolved, all eleven parity tests and fifteen MIPS instruction-suite tests pass under JIT-by-default.

The JIT’s overhead on programs without precompile syscalls is small (a fraction of a millisecond per call); on tendermint, which has approximately one syscall per several hundred microseconds of CPU work, the JIT delivers a $\sim 6\times$ speedup on the inner trace-mode execute loop. End-to-end on `reth`, where the GPU prover dominates wall time, the JIT speedup is within run-to-run noise: the executor speedup is invisible at the e2e level once the GPU prover is engaged.

4.10 Verifying key cache and program cache

Two host-side caches reduce repeated work across shards of the same workload. The *compose-pk* cache keeps the proving key for each compose-program shape resident on the GPU, keyed by program identity; on a shared-shape shard sequence this saves the per-shard preprocessed-trace generation and the device upload. The *program* cache keeps the compiled compose-program itself, keyed by arity. Together these eliminate the synthesis cost of repeated compose calls within a single `prove_core` run.

The caches are opt-in by default in development and on-by-default in production, controlled by a single `ZIREN_GPU_RESIDENCY` environment variable that selects among `host` (caches off), `hybrid` (default, conservative), and `full` (all caches on). Legacy per-cache variables are deprecation-warned but still honoured for backwards compatibility.

4.11 LogUp-GKR shape divergence between prover and recursion verifier

The largest piece of open architectural work in ZIREN 2.0 is the alignment of two LogUp-GKR proof types. The prover (in the `stark` crate) emits per-chip layer-based proofs whose serialisation walks the GKR circuit layer by layer; the recursion verifier (in the `recursion-circuit` crate) consumes per-shard round-based proofs whose serialisation walks the sumcheck stack round by round. The two were designed independently — the prover predates the migration, the recursion was written ahead of the eventual prover migration — and they encode the same protocol at different granularities.

Three unification approaches are open: migrate the prover to emit the recursion’s per-shard shape; add a per-shard adapter at proof-construction time; or add a per-chip verifier on the recursion side. The first is the cleanest architectural outcome but breaks wire compatibility with all previously-serialised proofs. The second is wire-compatible with the new shape but adds an extra reformat step per shard. The third is the most invasive on the recursion side but the least invasive on the prover. We are tracking this as a design decision to be made at the next major revision.

4.12 What Ziren 2.0 chose not to do

For clarity, we record several design directions that ZIREN 2.0 explicitly does *not* pursue.

- No use of the WHIR PCS family. An earlier exploration of WHIR as a default was abandoned in favour of BaseFold due to commit-bound workload performance on real Ethereum-block proving.
- No general-purpose runtime backend abstraction layer. The CPU and GPU paths share a high-level orchestration interface but diverge in implementation; we have deliberately not factored the difference into a trait abstraction, partly because the GPU-specific lifetime and stream-binding constraints do not map cleanly onto a single Rust trait.
- No retirement of the interpreter MIPS executor. The interpreter remains the fallback for unsupported opcodes (a small set: LWL/LWR, SWL/SWR, DIV/DIVU, MADD/MSUB and their unsigned variants, SEXT, INS, EXT, ROR, WSBH, all of SYSCALL) and for non-Linux-x86_64 hosts. Bit-exactness regression in the JIT is a soundness bug; the interpreter is a permanent defence.

5 Performance evaluation

5.1 Methodology

All multi-GPU measurements are taken on a single host equipped with eight NVIDIA RTX 5090 GPUs (one with 32 GB of visible memory each), 124 logical CPU cores, and 925 GB of system RAM. CPU-only measurements use a 16-core, 123 GB RAM x86_64 host running a Linux 5.x kernel. We use the JIT-by-default executor end-to-end through prove and verify, the GPU prover under the production residency profile (compose-PK cache on; program cache on; per-syscall trace sync on), and trace-generation worker pool sized at 16.

For each configuration we run at least three end-to-end measurements and report the median. Run-to-run variance on a shared GPU box is non-trivial: on contention-free runs we see $\pm 3\%$ wall-time variance; under contention we observe outliers as much as $3\times$ slower, which we discard from the reported medians per our experimental discipline. Verification runs end-to-end on every reported configuration; we do not measure prove-only timings.

We note explicitly that this paper does not report a head-to-head comparison against SP1 on identical workloads. The two systems target different guest ISAs (MIPS32r2 versus RV32IM), use different chip decompositions, and use different test-fixture programs; a fair comparison would require a non-trivial fixture-equivalence harness that is beyond our current scope.

5.2 End-to-end results on Ethereum execution-layer blocks

Our primary production workload is the proving of an Ethereum execution-layer block via `reth`, a Rust implementation of the Ethereum consensus client. The test fixture is a single Ethereum block plus its associated state proofs; the resulting MIPS execution trace is approximately 420 million cycles spread across 189 shards. Table 1 reports end-to-end timings at one, two, four, and eight GPUs.

The system scales from one GPU (159.2 s) to four GPUs (93.1 s) with a $1.71\times$ speedup. From four to eight GPUs the scaling plateaus: the core stage’s per-shard CPU trace-generation cost saturates at four workers, and the compress stage’s pipelined-arity choice scales 1:1 with GPU count up to the pool size. The shrink and wrap stages are single-GPU by design and contribute approximately 1.5 s irrespective of pool size.

Table 1: End-to-end (prove and verify) wall time on **reth** (Ethereum execution-layer block, ~ 420 M cycles, 189 shards) at varying GPU count. Median of three runs.

GPUs	Core (s)	Compress (s)	Shrink (s)	Wrap (s)	Total (s)
1	109.3	48.4	0.41	1.14	159.2
2	73.3	35.3	0.45	1.09	110.1
4	62.3	29.3	0.47	1.11	93.1
8	62.7	29.8	0.47	1.07	94.1

Table 2: Per-workload core-prove throughput on a CPU-only host (legacy FRI versus BaseFold + Jagged at rate 1/2 and rate 1/16). Wall time in seconds; speedup expressed as ratio of FRI to BaseFold wall.

Workload	Cycles	FRI (s)	BaseFold _{1/2} (s)	BaseFold _{1/16} (s)	Best speedup
fibonacci-1k	8 K	13.6	32.0	9.5	1.44 $\times$
json	9 K	56.4	18.4	29.8	3.07 $\times$
chess	12 K	61.4	23.5	34.8	2.62 $\times$
fibonacci-100k	700 K	384.6	366.6	461.3	1.05 $\times$

5.3 Per-workload throughput sweep

Table 2 reports the wall time of core proof generation across four representative workloads under three configurations: the legacy FRI baseline (now removed in the current revision), the BaseFold rate-1/2 development configuration, and the production rate-1/16 configuration with 16 bits of Poseidon22-based grinding.

The BaseFold stack delivers a 3.07 $\times$ improvement on commit-bound mid-size workloads (JSON parsing) and a 2.62 $\times$ improvement on the chess-engine workload. On the tiny Fibonacci workload, the per-MLE encoding fixed costs dominate, and the rate-1/2 configuration loses; the production rate-1/16 configuration recovers to 1.44 $\times$ over FRI. On the largest CPU-feasible workload (Fibonacci with 700 000 cycles), BaseFold at rate-1/2 is on par with FRI, while rate-1/16 pays a $\sim 20\%$ penalty for the 8 $\times$ larger codeword.

5.4 Out-of-memory cure

The legacy FRI prover on the multi-million-cycle workloads (Tendermint light-block verification at ~ 10 M cycles, an extended-fibonacci of 21 M cycles, and real Ethereum-block proving via **reth** and **geth**) exhausted memory on a 123-GB-RAM host, with peak resident set sizes of 107–112 GB during the FRI commit phase. The migration to BaseFold + Stacked + Jagged structurally cures this: per-MLE codeword encoding streams stripes of bounded size through the device DFT, so peak memory stays bounded by stripe size $\times$ blowup factor rather than whole-vector $\times$ blowup. Tendermint completes at a ~ 52 GB peak resident set; the extended Fibonacci completes at ~ 52 GB. Both were previously unprovable on the same hardware.

5.5 Compress stage decomposition

The compress stage is the next most expensive stage after core proving on the multi-GPU **reth** measurements. Its ~ 30 s wall time at four GPUs decomposes into the per-shard recursion-program

build, the dummy-witness construction in the dummy-VK-and-shard-proof generator, the per-shard Jagged-fingerprint computation, and the Merkle-path open. The dummy-proof port (§3.7) reduced startup pre-warm from 64.8 s to 2.0 s. The verifying-key cache (§3.6) absorbed a further ~ 3.3 s on a 4-GPU `reth` run via a $\sim 60\%$ hit rate across shape-equivalent shards.

5.6 JIT executor speedup

The JIT executor, measured in isolation on a 100 000-instruction pure-ALU register-arithmetic chain, completes in 1.83 ms with 18.3 ns per instruction. The interpreter under *simple* mode (no event recording) completes the same chain in 1.77 ms; the interpreter under *trace* mode (with event recording for prover consumption) requires 10.74 ms, 107.4 ns per instruction. The JIT thus delivers a $5.9\times$ speedup over the interpreter in the prover-relevant trace mode.

End-to-end on small workloads, the JIT loses to the interpreter due to per-call setup overhead (transpilation of the program and one-time mmap of the 4 GB guest memory bridge). On large workloads such as `reth` where the GPU prover dominates wall time, the JIT and interpreter timings are within run-to-run noise; both produce byte-identical execution traces.

5.7 Scaling analysis

The scaling characteristics observed across our workload sweep can be summarised as follows.

One-to-two GPU. A $1.45\times$ total speedup ($1.49\times$ core), reflecting near-linear core-stage scaling with two trace-generation workers per GPU.

Two-to-four GPU. A $1.14\times$ total speedup, with compress scaling from 35.3 s to 29.3 s (-17%) while core scales from 73.3 s to 62.3 s (-15%). The shrink and wrap stages are unchanged.

Four-to-eight GPU. A $1.02\times$ total speedup, effectively flat. The core stage’s CPU-side trace-generation pool saturates at four workers under the current per-shard state-mutex contention; further scaling requires profiling and refactoring of the trace-gen worker loop. The compress stage scales 1:1 with the GPU pool but its absolute floor is small enough that the gain is absorbed by the core stage’s plateau.

We anticipate that a follow-on revision profiling the per-shard state-mutex contention will unlock the four-to-eight scaling regime, restoring the linear curve out to eight GPUs.

6 Discussion

6.1 Why two prover paths persist

The continued maintenance of two structurally distinct prover paths (the SP1-aligned and the ZIREN-native dispatches; §4.1) reflects a deliberate design choice rather than an accident of incremental migration. Aligning the inner sumcheck algebra across paths is non-trivial work that involves both the prover-side polynomial-fold direction and the verifier-side challenge-evaluation reordering. Until that alignment is complete, attempting to run all production traffic through the SP1-aligned path would silently fail the recursion verifier at LogUp-GKR round one and beyond. The ZIREN-native dispatch, by contrast, has been validated end-to-end through every recursion layer up to BN254 wrap. We chose to accept the maintenance cost of two paths in exchange for a continuously-shippable production proof system.

The path-aware verifier dispatch (§4.6) is the intermediate accommodation: a single proof carries a fold-orientation tag, and verifiers can route to the appropriate algebra. This allows the system to ship correct proofs from either path immediately and to converge the algebras incrementally.

6.2 The cost of soundness invariants

A consistent thread through the implementation history is that the soundness-critical invariants (the per-chip-padded sum convention for witness sizing, the per-round zero-column padding for Jagged prefix sums, the canonical-shape fix for compose-tree caching) have been discovered through verifier failures rather than through up-front formal specification. Each invariant, once identified, is a small mechanical fix; identifying it requires either reading the SP1 reference carefully or tracing a verifier panic backward through several layers of abstraction.

This is, in our experience, a common pattern in zkVM engineering: the soundness invariants of a multi-protocol stack are not all written down in any single place, and they are not visible at the type system level (a witness of the wrong size still type-checks). A more rigorous discipline would emerge from a formal verification effort, but in the interim we lean on differential testing against SP1 and aggressive end-to-end verification on every change.

6.3 The verifying-key map and proof-shape stability

The verifying-key map mechanism (§3) is a pragmatic response to the difficulty of constructing a single recursion verifier that handles every possible program shape. By restricting the set of admissible shapes to those enumerated in the map, the prover can specialise the recursion program at compile time to the specific shape it expects.

This has consequences for the wire format of proofs: a proof carries an implicit reference to its verifying-key map entry, and a recipient must have the matching map to verify the proof. We treat the map as part of the protocol parameters, alongside the field choice and the security level; map regeneration is a process step that occurs at protocol version bumps and is documented as such.

6.4 Known limitations

In-circuit verifier gap under strict verifying-key enforcement. Under the `VERIFY_VK=true` mode, certain workloads still hit an inner-sumcheck-algebra divergence between the path the GPU prover takes and the path the in-circuit recursion verifier expects. We treat this as a soft deficiency rather than a soundness issue: the host verifier accepts the proofs, and the inner-sumcheck algebra port is an active work item. The proper resolution is the full alignment of the two paths discussed in §4.1.

Backend abstraction deferred. The CPU and GPU paths do not share a trait-level abstraction layer. We have not factored the difference into a runtime backend interface because the GPU-specific lifetime and stream-binding constraints do not map cleanly onto a single Rust trait without losing the ability to express the device-residency lifecycle. This decision should be revisited if a third backend becomes desirable (e.g., AMD HIP or distributed CPU clusters).

LogUp-GKR proof-shape divergence. The proof-shape divergence between the prover’s per-chip layer-based serialisation and the recursion verifier’s per-shard round-based serialisation (§4.11) is a documented architectural gap. The current production system bridges it via an in-tree adapter, but the cleanest long-term outcome is prover migration to the recursion shape. This is gated on a wire-format break that we plan to schedule alongside other protocol-version changes.

Multi-host distribution. The current system runs on a single host with up to eight GPUs. Distribution across multiple hosts is not yet implemented; the natural shape is to distribute shards across hosts in the core stage and aggregate the compress tree on a single host, but the engineering and bandwidth analysis is open.

Inner-sumcheck algebra port. The full port of SP1’s inner sumcheck folding to ZIREN’s tower implementation is the open work that gates a single unified prover path. The MLE-full-Lagrange-eval primitive is in-tree but not yet wired into the inner sumcheck round computation; this is the long pole of the convergence effort.

6.5 Future work

We identify several directions for future work, ranked by anticipated impact.

1. **Inner-sumcheck algebra alignment.** Closes the gap between the two prover paths and unblocks a single unified production verifier.
2. **Per-chip BaseFold commit.** The current commit materialises the dense polynomial one-shot during Jagged reduction; restructuring to per-chip commit (mirroring SP1’s per-chip jagged-basefold structure) eliminates the brief peak-memory window during sumcheck and unlocks even larger workloads.
3. **Profile-guided release of the four-to-eight scaling regime.** The per-shard state-mutex contention that bounds core-stage scaling beyond four GPUs is a known target; profiling and refactoring should restore the linear curve out to eight.
4. **Streaming BaseFold commit upstream.** The current commit holds the dense vector across the sumcheck reduction; a streaming variant would reduce the peak working set further, at the cost of some round-trip latency.
5. **Multi-host distribution.** Bandwidth analysis, work scheduling, and the failure-handling model for distributed shard proving.
6. **Formal verification of soundness invariants.** The per-chip padded-sum convention, the Stacked dimension invariant, and the fold-orientation tag are all candidates for formal specification and machine-checked proof.

7 Related work

7.1 Zero-knowledge virtual machines

The contemporary zkVM landscape is dominated by RISC-V-targeted systems descended from the architectural template of SP1 [18]. SP1 introduced the chip-decomposed AIR construction, the recursion-tree compression, and the BaseFold-plus-Jagged commitment stack that ZIREN 2.0 adopts. ZIREN’s history of incremental SP1 alignment, including the present work, reflects our commitment to leveraging the SP1 design where it is well-engineered and diverging only where the MIPS target or implementation constraints warrant.

RISC0 [16] is an alternative RISC-V zkVM that uses a different cryptographic stack (over a 32-bit Mersenne prime field, with a different sumcheck-based commitment). Cairo [17] is a STARK-based

virtual machine with a custom instruction set designed jointly with the prover’s algebraic constraints; its development informed the chip-decomposition style now common across the ecosystem.

Plonky3 [15] is not itself a zkVM but provides the polynomial commitment, lookup, and sumcheck building blocks that underlie much of the contemporary zkVM ecosystem, including ZIREN 2.0. Our **BaseFold**, **Stacked**, and **Jagged** implementations build on Plonky3’s lower-level primitives.

The use of MIPS as a guest ISA in ZIREN 2.0 differs from the RISC-V-dominant ecosystem. The closest precedent is the Optimism MIPS interpreter used for fault proofs in the Optimism bedrock and Cannon designs [12], though Cannon is a single-step interpreter rather than a zkVM. The choice of MIPS in our setting is driven by ISA-maturity considerations rather than fault-proof compatibility.

7.2 Polynomial commitment schemes

The base **BaseFold** construction [22] interleaves sumcheck rounds with Reed–Solomon codeword folds, achieving multilinear-PCS structure with the round complexity of FRI. The **Stacked** and **Jagged** wrappers used in this work were introduced in SP1’s `slop` crate family and have not, to our knowledge, been previously published in stand-alone form.

The legacy FRI baseline [3] remains in the literature as the workhorse of small-field STARK systems. Brakedown [9] represents an alternative line of work using tensor-product codes for the underlying linear commitment; it has not seen widespread zkVM adoption due to the overhead of the linear-code commit.

The WHIR family of multilinear-PCS constructions [1] was explored as a possible default in an earlier iteration of ZIREN. The performance characteristics on real Ethereum-block proving favoured **BaseFold** on commit-bound workloads, and the WHIR path has been removed from the production tree; we cite WHIR here as the closest contemporary alternative.

7.3 Lookup arguments and sumcheck

The **LogUp-GKR** lookup argument [13] used by ZIREN 2.0 is a GKR-style sumcheck-based realisation of the Habök log-derivative lookup [10]. The choice of sumcheck-based GKR over alternatives (Plookup [7], cq [5], Halo2 lookups) reflects the practical tractability of GKR descent in recursion circuits over small fields.

The classical sumcheck protocol [11] underlies essentially every primitive in our stack: **BaseFold**’s round descent, **Jagged**’s reduction, **LogUp-GKR**’s GKR layers, and zerocheck’s per-chip constraint discharge are all sumcheck-based.

7.4 Recursion and outer wrap

The recursion architecture (compose tree, shrink, wrap) is essentially that of SP1 and its prior incarnations in the Plonky2 [14] ecosystem. The outer Groth16 wrap via **gnark** [4] on the BN254 curve is the standard choice for on-chain Ethereum verification.

The in-circuit verifier port is the longest-running engineering effort in the system; we follow SP1’s organisation of the recursion verifier into transcript, **LogUp-GKR**, zerocheck, and **Jagged** phases, with module-by-module porting accounted for in our development logs.

7.5 GPU acceleration of STARK proving

The use of GPUs for STARK proving has become standard practice in the production zkVM space [19]. Our GPU prover design is closest in shape to SP1’s `sp1-gpu` architecture: device-resident

dense trace MLE, per-stream worker pool, Poseidon22-based Merkle commit on the device, and a query phase that opens via device-side Merkle paths. Where we diverge is in the work-stealing pool implementation, the verifying-key cache strategy, and the per-syscall trace synchronisation cadence; these choices were informed by measurements on the RTX 5090 family of GPUs and may not generalise to other hardware.

7.6 Just-in-time interpreters for zkVMs

The use of JIT compilation to accelerate the guest-execution path of a zkVM was pioneered (to our knowledge) by SP1’s `sp1-jit` [20], which targets RISC-V. Our MIPS JIT borrows the architectural template (`dynasm-rt` assembly emission, GPR pinning into x86 XMM register halves, build-time `cfg` gate on Linux `x86_64`) and adapts the register layout to MIPS’s HI/LO pair and branch-delay-slot semantics.

8 Conclusion

ZIREN 2.0 consolidates fourteen months of incremental migration into a coherent second-generation zkVM stack for the MIPS32r2 instruction set. Its principal architectural changes from the previous generation are: a hierarchical **BaseFold** + **Stacked** + **Jagged** polynomial commitment construction that structurally cures the memory-pressure failures of the legacy FRI prover on multi-million-cycle workloads; a `sumcheck`-binding rewrite of the soundness role previously discharged by auxiliary commitments; a GPU prover with device residency, per-slot streams, a verifying-key cache, and pipelined compress that delivers a near-linear speedup curve up to four GPUs; and a JIT-compiled MIPS executor that is bit-exact against the reference interpreter.

We report end-to-end production-relevant timings on an Ethereum execution-layer block of 420 million MIPS cycles (94.1 seconds at eight GPUs, prove and verify, through Groth16 BN254 wrap), and we discuss the subtle soundness invariants and integration discipline that were essential to bringing the system to this performance level. We are honest about the remaining gaps in the in-circuit verifier’s algebraic alignment between the two prover paths, the proof-shape divergence in `LogUp-GKR` between prover and recursion verifier, and the four-to-eight GPU scaling plateau.

ZIREN 2.0 is in production deployment and we anticipate continued incremental development along the directions sketched in §6. Source code is released under permissive licences in the ZIREN and `ziren-gpu` repositories.

Acknowledgements

We thank the SP1 team at Succinct Labs for the publicly-available reference implementation and the architectural template on which much of ZIREN 2.0 is based. We thank the Plonky3 team for the polynomial-commitment, lookup, and `sumcheck` primitives. Specific implementation work in ZIREN 2.0 draws inspiration and code patterns from these projects; the system’s deficiencies are our own.

References

- [1] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification. *Cryptology ePrint Archive*, Paper 2024/1586, 2024.

- [2] David A. Barrington. Bounded-Width Polynomial-Size Branching Programs Recognize Exactly Those Languages in NC^1 . In *Journal of Computer and System Sciences*, volume 38, pages 150–164, 1989.
- [3] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed–Solomon Interactive Oracle Proofs of Proximity. In *Proceedings of the 45th International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 14:1–14:17, 2018.
- [4] Consensys. gnark: A fast zk-SNARK library in Go. <https://github.com/Consensys/gnark>, 2024.
- [5] Liam Eagen, Dario Fiore, and Ariel Gabizon. cq: Cached quotients for fast lookups. Cryptology ePrint Archive, Paper 2022/1763, 2022.
- [6] Amos Fiat and Adi Shamir. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology — CRYPTO ’86*, pages 186–194, 1987.
- [7] Ariel Gabizon and Zachary J. Williamson. plookup: A simplified polynomial protocol for lookup tables. 2020.
- [8] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. Delegating Computation: Interactive Proofs for Muggles. In *Proceedings of the 40th ACM Symposium on Theory of Computing (STOC)*, pages 113–122, 2008.
- [9] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad Wahby. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. Cryptology ePrint Archive, Paper 2021/1043, 2021.
- [10] Ulrich Habök. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Paper 2022/1530, 2022.
- [11] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic Methods for Interactive Proof Systems. In *Journal of the ACM*, volume 39, pages 859–868, 1992.
- [12] Optimism Foundation. Cannon: A MIPS interpreter for Optimism fault proofs. <https://github.com/ethereum-optimism/optimism>, 2023.
- [13] Shahar Papini and Ulrich Habök. Improving logarithmic derivative lookups using GKR. Cryptology ePrint Archive, Paper 2023/1284, 2023.
- [14] Polygon Zero. Plonky2: Fast recursive arguments with PLONK and FRI. Whitepaper, 2022.
- [15] Polygon Zero and contributors. Plonky3: A toolkit for STARKs over small fields. <https://github.com/Plonky3/Plonky3>, 2024.
- [16] RISC Zero, Inc. RISC0: A general-purpose zero-knowledge virtual machine. <https://www.risczero.com>, 2024.
- [17] StarkWare Industries. Cairo: A Turing-complete STARK-friendly CPU architecture. <https://www.cairo-lang.org>, 2021.
- [18] Succinct Labs. SP1: A performant, open-source, contributor-friendly zkVM. <https://github.com/succinctlabs/sp1>, 2024. Accessed 2026.

- [19] Succinct Labs. sp1-gpu: GPU prover for SP1. <https://github.com/succinctlabs/sp1>, 2025. Component of the SP1 repository.
- [20] Succinct Labs. sp1-jit: Just-in-time RISC-V executor for SP1. <https://github.com/succinctlabs/sp1>, 2025. Component of the SP1 repository.
- [21] The ZKM Project. Ziren: A zero-knowledge MIPS virtual machine. <https://docs.zkm.io>, 2025.
- [22] Hadas Zeilberger, Binyi Chen, and Ben Fisch. BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes. Cryptology ePrint Archive, Paper 2023/1705, 2023. URL <https://eprint.iacr.org/2023/1705>.